

ENGINEERING AUDIT

Technical audit — the core of the Timesheet app

A code-level review of the six subsystems that carry the day-to-day work: job identity and resolution, territory (country) resolution, the timesheet entry pipeline, the Wrike sync and cache, the Management surface, and the auth / data layer. Every finding cites the file and line it was read from.

1 How the audit was run

Six audit agents worked in parallel, one per subsystem. Each read the actual source (not a summary of it) and returned findings that had to be grounded in a `file:line` reference with a concrete failure scenario. The seven findings rated **critical** or **high** were then re-read by a second, adversarial pass — an independent agent re-walked the cited code path for each one and had to confirm, refute, or mark it plausible. **All seven were confirmed.** That second pass is the difference between a list of suspicions and a list of things that actually fail.

Severity	Count	Meaning
CRITICAL	1	Security / wholesale data risk — anyone with the app can reach the data layer.
HIGH	6	Wrong result or lost work under common conditions.
MEDIUM	18	Wrong result on an edge case, or a fragile path that is easy to trip.
LOW	16	Inconsistency, observability, or a latent risk that does not bite today.

Scope. This is a static, code-read audit. It did not run the app, touch the database, or call the live Wrike or Supabase APIs — so a finding confirmed here is confirmed in the code, not in live data. Where a scenario depends on data (e.g. “a user has 1,200 rows”), the code path that causes the failure is verified; the trigger is the realistic part.

2 The seven that matter

Everything below is a real, walkable failure in the code. Start here.

CRITICAL RLS write policies are wide open; the admin gate is UI-only

`supabase/schema.sql:509`

Summary. `jobs`, `positions`, `job_categories`, `job_departments`, `project_descriptions`, `job_sync_kept`, `board_now`, `wrike_tasks_cache`, `wrike_sync_meta` and the campaign/canvas/dooh tables all carry `for all to authenticated using (true) with check (true)` — and every visitor becomes an `authenticated` user via `signInAnonymously()` (`src/lib/supabaseClient.js:21`) against the anon key embedded at `supabaseClient.js:5`.

Failure. Anyone who opens the app, signs in anonymously, and knows the anon key can `PATCH / DELETE` on `/rest/v1/jobs`, set `positions.hourly_rate = 0`, or wipe `board_now` directly. `MANAGEMENT_IDS` gates the React page (`Management.jsx:6041`, `App.jsx:460`), never the database. The *only* policy that enforces the admin allowlist (`KUAWDLVN`, `KUAQT4JC`) is `profiles_write` (`schema.sql:516`).

HIGH Scan keeps only the first folder per XY code — an archived copy can shadow the live one

`src/lib/wrikeCampaign.js:345`

Summary. `scanStudioJobNumbers` de-dups by code with a first-writer-wins `seen` set *before* computing ancestry, so the folder the scan describes is whichever one Wrike happens to list first.

Failure. Code `XY025716` on both a live folder (`SONY > Focker In-Law`) and an `_Archive` copy. If the archive copy is enumerated first, `seen.has(code)` skips the live folder entirely; the job is reported *archived*, gets film “Archive”, and vanishes from the default “Active only” view — even though the multi-parent machinery is supposed to keep live copies alive. It only walks one folder’s own parents; it cannot compare two folders that carry the same code.

HIGH The scan re-creates the mangled job numbers it was meant to fix`src/lib/wrikeCampaign.js:359`

Summary. Any folder title containing " : " is copied *verbatim* into `job_number` — so a title whose code position carries a suffix, like `The Odyssey : XY025716_INTL_DIGITAL_OUTDOOR_CAMPAIGN_MARKETS, INT - DOOH...`, is re-registered as-is. That is the exact “self-perpetuating” shape `wrikeHelpers.js:16-20` says must never enter the code position.

Failure. The 23 legacy rows documented at `wrikeHelpers.js:17-18` came from folders still named this way. On the next scan the mangle sits back in the code position; pull-time `resolveJobNumber` and `getJob` adopt it verbatim. The fix that hardened the pull path was never applied to the scan path.

HIGH Logged rows are stamped with the logging date, not the selected day`src/hooks/useTaskActions.js:90`

Summary. `handleLogTask` writes `dayOfWeek: selectedDay` but `date: new Date().toLocaleDateString('en-GB')` — always today. `useTasks.toDb` derives `work_date` from `date` (`useTasks.js:38`), and the week view keeps only rows where `workDate >= weekStart` (`useTasks.js:147-150`).

Failure. On Sunday 2026-08-09, selecting *Monday* and logging 1h writes `dayOfWeek=Monday`, `date=09/08/2026`, `work_date=2026-08-09`. When Monday arrives, `weekStart` moves to 2026-08-10, the filter excludes the row, and the Monday entry vanishes from the grid while staying in the DB. Within a week, backfilling Monday on a Wednesday stores *Wednesday's* date on a Monday row. The Legacy modal has the same mismatch (`LegacyTimesheets.js:281-323` vs `useLegacyRows.js:65`).

HIGH A purge with a stale dictionary permanently deletes still-relevant tasks`src/hooks/useWrikeCache.js:651`

Summary. `sync()` purges any task that fails `filterToMotionTeam`, but that filter matches assignees and DIGITAL folders against a `contact_dictionary` that is only refreshed every 24h (`useWrikeCache.js:604`) and is never checked for freshness.

Failure. Wrike adds a new Motion member Pat; a task is reassigned to Pat. With a 20h-old contact dict, `motionTeamShortName(contactDictionary[patId])` is undefined, the task fails the filter, lands in `droppedIds`, and is `DELETE`d from the shared `wrike_tasks_cache` for the whole team. Its `updatedAt` is now older than the advanced `last_synced_at`, so no later delta sync re-fetches it. The task stays gone until someone presses the manual sync button.

HIGH StudioAnalytics drops every CSV-imported row from the hours charts`src/components/StudioAnalytics.jsx:47`

Summary. `parseDdMmYyyy` splits on `/`, so an ISO date (`2026-01-05`) parses to `null`. The CSV importer writes exactly that shape (`worker/index.js:187-198`, documented in migration `20260810090000`), while everything written in the app is `09/08/2026`.

Failure. The “Hours logged” KPI (`hours30`) and “Hours logged per week” silently drop every imported row, while “Hours by client” and “Hours by department” do no date parse and include them — so the four charts disagree and the headline number undercounts imported time. The authoritative `work_date` column is selected by the feed yet ignored.

HIGH Timelog dedup reads at most 1000 rows`src/lib/supabaseClient.js:87`

Summary. `fetchExistingTimelogIds` does a plain `.select()` with no `.range()` pagination — the exact failure mode `selectAll` was written to fix (its own comment, `lines 42-47`). PostgREST caps at 1000 rows with no error.

Failure. A user with 1,200 timelogged task rows gets 200 ids missing from the dedup set. A later pull of one of those timelogs (`useTaskActions.js:192/216`, `LegacyTimesheets.js:1413`) re-inserts a duplicate row carrying the same `wrike_timelog_id`.

3 Job identity & resolution

`jobKey` (`wrikeHelpers.js:56-60`) reduces any label — bare code, suffixed folder title, or canonical `Film : XY025716, Desc` — to its first `XY\d{5,6}` match. `useJobLookup` keys the whole book by that key, keeps the richest row on a tie, and `ensureJob` only ever gap-fills blank fields. The scan walks the folder tree, derives film/client/region by climbing ancestry, and a unique expression index (`substring(job_number from 'XY\d{5,6}')`, `schema.sql:434`) enforces one row per code. This slice carries two of the seven confirmed highs (the shadowing archive copy and the re-mangled canonical branch) plus the medium risks below.

Sev	Finding	Where	What happens
MED	<code>resolveJobNumber</code> substring match returns the wrong job when one code is a digit-prefix of another	<code>wrikeHelpers.js:68</code>	<code>.includes(rawXy)</code> is unanchored, so a 5-digit code <code>XY02571</code> matches a book row whose 6-digit code <code>XY025716</code> merely starts with it, and that row's film/code are written for the wrong job.
MED	Scan and pull disagree on the client label for non-Universal studios	<code>wrikeCampaign.js:354</code>	The scan writes "Sony Pictures International", pull-time writes "Sony Pictures"; client-grouped analytics split one studio across two labels.
MED	Shared-folder path ties are left to Wrike list order	<code>wrikeCampaign.js:325</code>	A folder shared into both <code>UNIVERSAL</code> and <code>UNIVERSAL AUSTRALIA</code> picks whichever parent lists first — client and <code>AUS -</code> prefix can flip between rescans.
MED	<code>ensureJob</code> optimistic add masks the DB's authoritative row after a 23505 collision	<code>useJobLookup.js:89</code>	Two tabs race to register a code; the loser keeps its raw suffixed string in the local map for the whole session, so pulls re-write the mangle and skip the gap-fill.
MED	<code>jobs_job_code_key</code> keys on the <i>first</i> code-like substring anywhere in the label	<code>schema.sql:434</code>	A free-text internal job like <code>Repro XY25000 spec</code> claims key <code>XY25000</code> ; the real job then can't insert. Case divergence (index is POSIX, <code>jobKey</code> is <code>/i</code>) lets case-variants coexist in the DB while collapsing in JS.
LOW	Year folder becomes the film title when a job sits directly under a year folder	<code>wrikeCampaign.js:287</code>	<code>SONY > 2026 > XY025716_...</code> produces film <code>"2026"</code> and row <code>"2026 : XY025716, Germany Launch Assets"</code> .

4 Territory (country) resolution

The resolver reads a country only where someone deliberately put one, in order: last tokens of the task name → parent task name → folder climb → Country custom field matched by id (`countryCodes.js`, `countryField.js`). Alias precedence is territory names < `REGION_ALIASES` < `MAGI_MARKET_CODES` < agreed exceptions. The design rule ("read, never guessed") holds up well; the findings here are the places the rule is bent by accident.

Sev	Finding	Where	What happens
MED	<code>REGION_ALIASES</code> leaks Maldives/Nepal — non-roster territories reachable from a suffix	<code>countryCodes.js:90</code>	<code>CODE_LOOKUP</code> adds <code>REGION_ALIASES</code> without the roster check <code>setRuntimeAliases</code> applies, so <code>_MDV</code> resolves to "Maldives" — a value the picker can't re-select and the company-timesheet paste doesn't tick.
LOW	Greedy pair-rejoin manufactures a country from two unrelated trailing tokens	<code>countryCodes.js:206</code>	<code>Master_US_A</code> → USA (<code>US</code> + <code>A</code>), <code>Slovenia_SI_N</code> → Slovenia + Singapore; a market nobody wrote is read.
LOW	Curated <code>TT</code> alias cannot re-enable TT at the end of a name	<code>countryCodes.js:197</code>	<code>SUFFIX_BLOCKED</code> runs before <code>RUNTIME_ALIASES</code> , so the documented escape hatch for Trinidad doesn't work in the one place TT is blocked.
LOW	MAGI's <code>CA</code> = Canadian-French shadows <code>REGION_ALIASES</code> ' <code>CA</code> = Canada	<code>constants.js:318</code>	Documented, but a <code>_CA</code> suffix never means Canada; only the full word <code>_CANADA</code> does.
LOW	Country-field reader splits on <code>/</code> and can never read the roster territory "Latam / Las"	<code>countryCodes.js:301</code>	<code>[,;/;]+</code> tears <code>Latam / Las</code> into <code>['Latam', 'Las']</code> ; neither resolves, so the field yields <code>[]</code> .
LOW	Loose <code>/^country/</code> pin fallback can bind a non-Country field	<code>countryField.js:85</code>	Absent an exact "Country" title, a "Country of Origin" field would be pinned and trusted. Latent — this account has a real Country field.

5 Timesheet entry pipeline (Tracker + Legacy)

All rows for both surfaces go through `useTasks` — the single store. The DB keeps time as `H:MM` text while memory keeps exact seconds; `toDb` / `fromDb` and `updateTask` convert via the shared `parseTimeToSeconds` / `secondsToHM`. Dates live in `date` (text) plus a derived `work_date` used for the week filter. The confirmed high (logging-date vs selected-day) sits at the top of this stack; the rest are rounding and representation mismatches.

Sev	Finding	Where	What happens
MED	<code>secondsToHM</code> rounds to the nearest minute on write while in-memory seconds stay exact	<code>timeHelpers.js:44</code>	A 59:59 timer stores <code>1:00</code> but the session total and export report 3599s; after reload the row parses back to 3600s — one row, two durations.
MED	Film title truncated at the first colon in Tracker log/edit paths	<code>useTaskActions.js:79</code>	<code>/^([^\:]+)\s*:/</code> stops at the first colon, so <code>Paw Patrol: The Dino Movie : XY025793, ...</code> registers film “Paw Patrol” — the codebase documents the split must be on <code>" : "</code> .
MED	Tracker Wrike pull derives weekday from a UTC-parsed <code>trackedDate</code>	<code>useTaskActions.js:194</code>	West of Greenwich, a Saturday log lands in Friday's column — the Legacy pull already fixed this (<code>LegacyTimesheets.js:1414-1419</code>); the Tracker path never got it.
LOW	Legacy grid saves decimal picks as <code>H:MM</code> , so the cell value never matches a dropdown option after reload	<code>useTasks.js:240</code>	Pick <code>0.25</code> , it saves <code>0:15</code> ; next load no option matches, nothing is highlighted.
LOW	Manual hours input truncates decimals via <code>parseInt</code>	<code>useTaskActions.js:66</code>	<code>1.5</code> hours logs 1:00, silently losing 30 minutes; the <code>TimerPanel</code> preview mirrors the truncation.
LOW	<code>useLegacyRows</code> pins <code>weekStart</code> at mount	<code>useLegacyRows.js:50</code>	A tab left open across midnight Monday keeps rendering last week's rows (and the empty week looks populated) until reload.

6 Wrike sync & cache

A team-shared mirror of Wrike tasks: hydrate from IndexedDB, delta-pull rows past a local cursor with 24h overlap, enrich, upsert, mirror, purge what dropped out of relevance, and stream live updates through a Supabase Realtime subscription. The confirmed high (stale-dictionary purge) is the expensive one here — it is the one path that deletes shared data on a false negative. The rest are windowing and error-handling gaps.

Sev	Finding	Where	What happens
MED	Webhook events for deleted tasks never purge them from any cache	<code>useWrikeCache.js:834</code>	<code>TASK_DELETED</code> webhooks are delivered but the handler never reads <code>event_type</code> ; a deleted task keeps rendering and keeps being served forever.
MED	Mount hydrate treats any query error as end-of-data and proceeds silently	<code>useWrikeCache.js:315</code>	A transient network error on the first page breaks the pull loop; on a cold start the UI renders zero tasks with no error and no retry.
MED	Lifetime stats record zeros on a non-OK API response and cache them for 24h	<code>useWrikeUser.js:77</code>	<code>response.ok</code> is never checked; an expired token shows “0 completed tasks” with no error, and the background refresh is suppressed for a day.
MED	<code>sync()</code> advances <code>last_synced_at</code> to wall-clock <i>now</i> , skipping mid-sync updates	<code>useWrikeCache.js:738</code>	A task edited during the fetch window is permanently invisible to every later delta sync until it changes again (the local cursor keeps an overlap; this cursor does not).
MED	Hub-subtask protection is computed before pagination-stripped ids are restored	<code>useWrikeCache.js:652</code>	When a launch hub lands past page 1, its <code>subTaskIds</code> are dropped, so its per-market subtasks aren't protected and can be purged.
LOW	Webhook flush drops the batch on an unhandled rejection	<code>wrikeWebhookSubscription.js:19</code>	<code>pendingIds</code> is cleared before the awaited writes; a rejection loses the whole burst with no retry.
LOW	Hydrate merge retains a stale MATRIX table when it was legitimately removed	<code>useWrikeCache.js:330</code>	Deleting the <code><table></code> from a MATRIX task's description never clears the stored one; the old rows keep rendering.
LOW	State-updater callback performs side-effecting writes	<code>useWrikeCache.js:755</code>	<code>setTasks</code> updater issues Supabase updates and IndexedDB saves; under <code>StrictMode</code> this runs twice and can desync the two stores on a failed write.
LOW	<code>fetchWrikeOAuthStatus</code> reports “disconnected” on any transient failure	<code>wrikeApi.js:16</code>	A 503 makes the UI hide/disable Wrike features as if the user disconnected; no retry, no classification.

7 Management surface (Job Book, scans, analytics)

The Management surface exports `JobsSetupSection` / `JobBookSection` / `JobsFeedSection` (the standalone Job Book is a three-tab shell over them). The Studio Job Scan reconciles book rows against the folder scan, classifies each code as correction/fixable/kept/redundant, inserts new codes in 200-row chunks, and rewrites or deletes on confirm. StudioAnalytics reads the studio timesheet through the worker feed but reads the Wrike-side tiles from the motion-filtered shared cache. One confirmed high (CSV dates) plus the reconciliation gaps below.

Sev	Finding	Where	What happens
MED	Campaign progress treats custom-status work (In Progress/Backlog) as neither done nor active	<code>StudioAnalytics.jsx:508</code>	The status field only ever holds base statuses; a film with 50 Completed + 50 "In Progress" tasks shows 100% complete and "in production" false.
MED	"Whole studio" Wrike tiles are computed from the motion-team-filtered cache	<code>StudioAnalytics.jsx:466</code>	Active tasks, completed-per-month and campaign progress undercount any work that isn't DOOH/DINTH/MATRIX/print or motion-assigned — the "one slice shown as the whole" class of bug the timesheet side already fixed.
MED	Scan "Add to Job Book" chunked insert is non-transactional	<code>Management.jsx:3568</code>	A mid-run failure leaves earlier chunks committed; the retry then re-inserts them and fails on the unique index, with no way to resume cleanly.
MED	Job Book edit leaves <code>job_number</code> stale, and the scan's Fix silently overwrites the user's description edit	<code>Management.jsx:3688</code>	Editing the Project Description doesn't rebuild the embedded description in <code>job_number</code> ; the next scan sees a disagreement and rewrites both back to the folder's wording.
MED	Film-picker activity ordering reads jobs with a single unpaginated query	<code>Management.jsx:2393</code>	Past 1,000 jobs, only an arbitrary first-1000 subset drives "most recently worked-on film first" (migration <code>20260809190000</code> already counts 949).
LOW	Kept disagreements still appear in the "N entries disagree" banner every scan	<code>Management.jsx:3770</code>	Kept rows are only excluded from the Fix-run subset, not from <code>corrections</code> , so the banner keeps nagging — <code>job_sync_kept</code> 's documented "future scans skip" promise isn't implemented.

8 Auth, access & data layer

The client boots an anonymous Supabase session and stamps the Wrike user ID into JWT `user_metadata`; RLS reads that. Per-user rows (`tasks`) are scoped by `wrike_user_isolation`; profiles by `profiles_read` / `profiles_write`. This slice holds the audit's only critical: the anonymous-everyone-is-authenticated model plus wide-open write policies means the admin allowlist is enforced in the UI only.

Sev	Finding	Where	What happens
MED	<code>selectAll('job_sync_kept', ...)</code> orders by a non-existent <code>id</code> column and silently returns empty	<code>Management.jsx:3445</code>	<code>selectAll</code> always applies <code>.order('id')</code> , but <code>job_sync_kept</code> 's primary key is <code>code</code> ; the query 400s, is caught, returns <code>[]</code> , and every previously-kept code is re-flagged. The Keep feature is currently write-only across reloads.
LOW	AdminModal issues one exact-count query per profile (N+1) and an unpaginated profiles read	<code>AdminModal.jsx:47</code>	~40 profiles = 41 round trips; past 1,000 profiles the team list silently truncates.
LOW	Admin UI gate and badge only recognize <code>KUAWDLVN</code> , not the second <code>MANAGEMENT_IDS</code> member	<code>App.jsx:442</code>	<code>KUAQT4JC</code> can write profiles (RLS allows it) but never sees the Admin button or badge — UI and data layer disagree on who is admin.

9 Reading the severity

Nothing here requires “this won’t happen”. The two that look worst are the RLS posture (critical) and the logging-date stamp (high) — the second is something a real user hit the week this audit ran (Sunday → Monday). The purge and the scan-shadow findings both silently destroy or hide data and have no automatic recovery. The cheapest wins are the date-stamp fix, the `job_sync_kept` ordering fix, and switching `fetchExistingTimeLogIds` to the existing `selectAll`.

Engineering audit of the Timesheeteer app · Six subsystems, six read-only audit agents, seven adversarial verification passes — all seven confirmed. Companion to job-data-technical.pdf, which describes the intended behaviour of the resolution pipeline. Generated 2026-08-10.